

FORMAL LANGUAGES AND COMPILERS

prof.s Luca Breveglieri and Angelo Morzenti

Exam of Tue 4 SEPTEMBER 2018 - Part Theory

LAST + FIRST NAME:

(capital letters please)

MATRICOLA:

(or PERSON CODE)

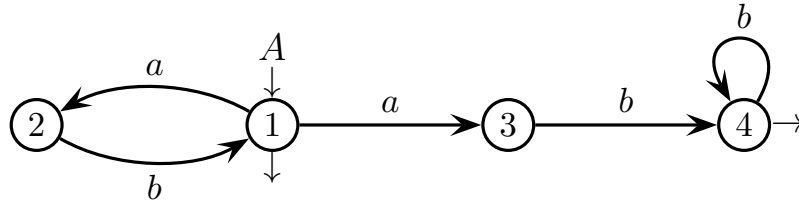
SIGNATURE:

INSTRUCTIONS - READ CAREFULLY:

- The exam is in written form and consists of two parts:
 1. Theory (80%): Syntax and Semantics of Languages
 - regular expressions and finite automata
 - free grammars and pushdown automata
 - syntax analysis and parsing methodologies
 - language translation and semantic analysis
 2. Lab (20%): Compiler Design by Flex and Bison
- To pass the exam, the candidate must succeed in both parts (theory and lab), in one call or more calls separately, but within one year (12 months) between the two parts.
- To pass part theory, the candidate must answer the mandatory (not optional) questions; notice that the full grade is achieved by answering the optional questions.
- The exam is open book: textbooks and personal notes are permitted.
- Please write in the free space left and if necessary continue on the back side of the sheet; do not attach new sheets and do not replace the existing ones.
- Time: part lab 60m - part theory 2h.15m

1 Regular Expressions and Finite Automata 20%

1. Consider the nondeterministic automaton A over the two-letter alphabet $\Sigma = \{ a, b \}$:



Answer the following questions:

- (a) Find the shortest strings of language $L(A)$ from length 0 (included) to length 2 (included) and write their computations.
 - (b) Through the Berry-Sethi method (*BS*), find a deterministic automaton A' equivalent to automaton A and minimize it if necessary. Verify the correctness of (the minimal version of) A' by means of the strings found at point (a).
 - (c) Through the node elimination method (*BMC*), find a regular expression R that generates language $L(A)$. Verify the correctness of R by means of the strings found at point (a).
 - (d) Design an automaton A'' that accepts language $\overline{L(A)}$, i.e., the complement of language $L(A)$. Counterverify the correctness of A'' by means of the strings found at point (a) (all of them have to be rejected).
 - (e) (optional) It is well known that in some cases a nondeterministic automaton may have fewer states than the equivalent deterministic minimal one. Can you find a (nondeterministic) automaton that accepts language $L(A)$ and has fewer states than the minimal automaton A' ? Explain your answer, whatever it is.
-

2 Free Grammars and Pushdown Automata 20%

1. Consider the following language L_1 over the two-letter alphabet $\Sigma = \{ a, b \}$:

$$L_1 = \{ a^n b^+ a^m \mid m \geq 0 \text{ and } \exists k \quad k \geq 0 \wedge n = m + 2k \}$$

For instance it holds:

$$a^3 b^2 a \in L_1 \qquad a^3 b^2 a^2 \notin L_1 \qquad a b^2 a^3 \notin L_1$$

Answer the following questions:

- (a) Write a *BNF* grammar G_1 , *not ambiguous*, that generates language L_1 .
(b) Check the correctness of grammar G_1 by drawing the syntax tree of the valid sample string:

$$a^3 b^2 a$$

- (c) Now consider language L_2 , mirror image of language L_1 , defined as follows:

$$L_2 = \{ a^m b^+ a^n \mid m \geq 0 \text{ and } \exists k \quad k \geq 0 \wedge n = m + 2k \}$$

Write a *BNF* grammar G , *not ambiguous*, that generates language $L = L_1 \cup L_2$.

- (d) (optional) Argue that grammar G_1 is not ambiguous.
-

2. Consider a simplified programming language, which consists of a list of assignments that feature the following syntax:

- A program is a possibly empty list of assignment statements.
- The assignment is denoted in a kind of *postfix* form, as follows:

`elem expr :=`

where “:=” (colon equal) is the postfix assignment operator and `expr` is the arithmetic expression to be assigned to element `elem`, as explained below.

- The expression `expr` is of the type sum-of-products (2-levels). It uses the arithmetic operators “+” and “*” (addition and multiplication), which are binary, i.e., with two operands. The expression is denoted in *prefix* form, thus without parentheses, as follows (on the right you can see the infix notation):

prefix form:	infix form as a comment:
<code>+ expr expr</code>	<code>expr + expr</code>
<code>+ + expr expr expr</code>	<code>expr + expr + expr</code>
<code>+ expr * expr expr</code>	<code>expr + expr * expr</code>

- The elements `elem` that can appear in an assignment (where it makes sense) and in an expression are the following:
 - a numerical constant, schematized by terminal `c`
 - a named variable, schematized by terminal `v`
 - an array element (one- or more-dimensioned), with the following syntax:

`vect ( list_of_expressions )`

where the expressions in the list (non-empty) are separated by “,” (comma), and the array name is schematized by terminal `vect`

Here is a short sample program (on the right there is the usual infix representation):

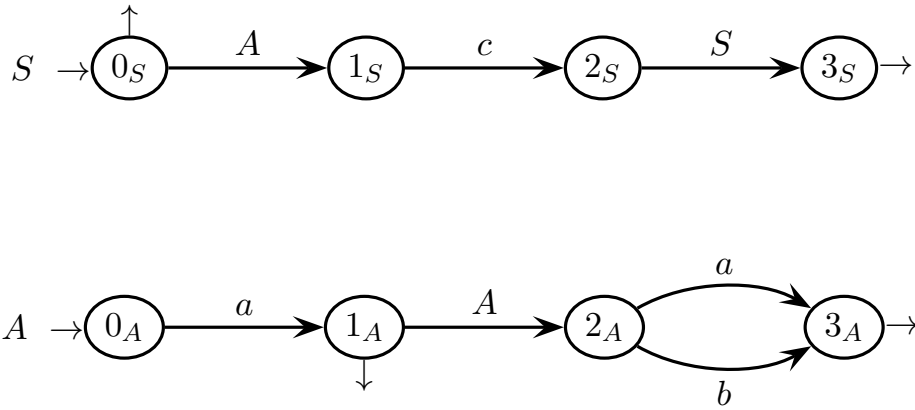
program:	comment (infix form):
<code>v c :=</code>	<code>v := c</code>
<code>v + v c :=</code>	<code>v := v + c</code>
<code>v + + c v v :=</code>	<code>v := c + v + v</code>
<code>v + * v v v :=</code>	<code>v := v * v + v</code>
<code>v vect (* v v) :=</code>	<code>v := vect (v * v)</code>
<code>vect (c, + v v) c :=</code>	<code>vect (c, v + v) := c</code>

Notice that the assignment operator “:=” also plays the role of statement terminator, to separate consecutive statements.

Write a grammar, possibly of *EBNF* type, not ambiguous, that generates the language described above. To test your solution, please draw the syntax tree for the last two assignments in the sample program.

3 Syntax Analysis and Parsing Methodologies 20%

1. Consider the following grammar G , represented as a machine net over the three-letter terminal alphabet $\Sigma = \{a, b, c\}$ and the two-letter nonterminal alphabet $V = \{S, A\}$ (axiom S):



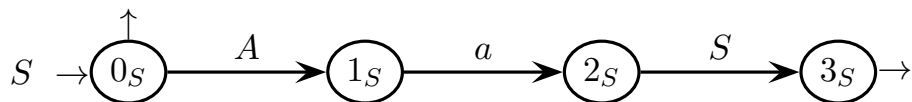
Answer the following questions (use the figures / tables / spaces on the next pages):

- (a) Draw a portion of the pilot of grammar G , sufficient to prove that grammar G is not of type $ELR(1)$. Highlight any conflict that is present in that pilot portion.
- (b) Determine whether grammar G is ambiguous or not, and provide a suitable explanation.
- (c) Draw all the guide sets on the net of grammar G (shift arcs, call arcs and exit arrows), determine whether grammar G is of type $ELL(1)$ and justify your answer.
- (d) Through the Earley algorithm for grammar G , analyze the valid string below:

$a a b c$

Draw the syntax tree and show which paths the net machines use to recognize.

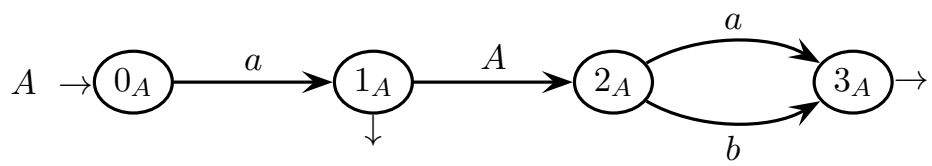
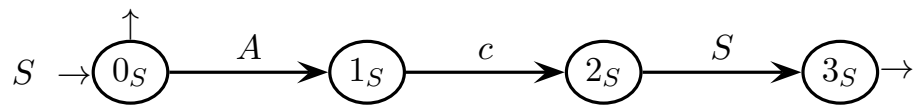
- (e) (optional) Consider a variant of the above grammar G , where machine M_S is modified as shown below, while machine M_A is unchanged:



Is this new grammar $ELR(1)$? Is it nondeterministic? Is it ambiguous? Explain your answer (hint: you may refer, as a significant example, to string $a a a a$).

here draw the pilot of grammar G – question (a)

here draw the call arcs and write all the guide sets of grammar G – question (c)



Earley vector of string $a\,b\,c$ (to be filled in) – question (d)
(the number of rows is not significant)

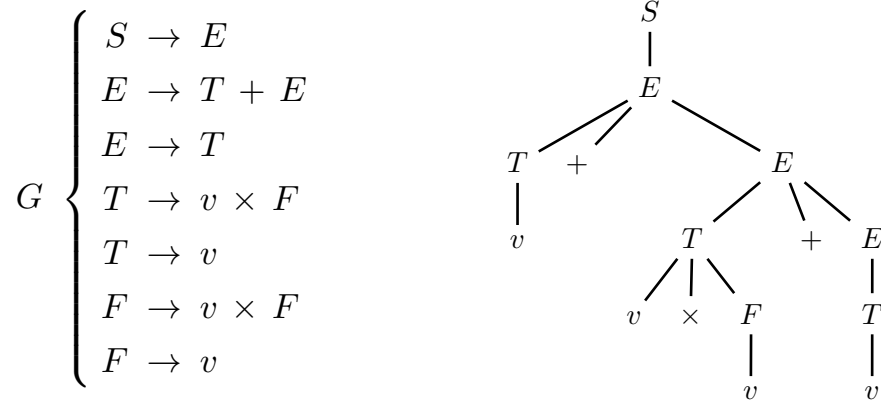
[illegible]

space for answering questions (b) and (e)

more space for the continuation of any exercise if necessary

4 Language Translation and Semantic Analysis 20%

1. Consider the following *BNF* non-ambiguous source grammar G (axiom S):



Grammar G generates two-level arithmetic expressions without parentheses, e.g., v , $v + v$, $v \times v$, $v + v \times v + v$, with variables and numbers schematized by terminal v . See the sample tree above on the right.

Answer the following questions (and do not change the source grammar G):

- (a) Consider a translation function τ_1 that transforms addition and multiplication into summatory $\sum$ and productory $\prod$, respectively:

$$\tau_1(v + v \times v + v) = \sum v, \prod v v, v \quad \text{and} \quad \tau_1(v) = \sum v$$

where a comma separates the addends of summatory $\sum$.

Write a syntactic scheme (or grammar) G_1 that models translation τ_1 . Draw the translation tree of the sample expression $v + v \times v + v$.

- (b) Assume that the element v is the same object (variable or number) wherever it occurs in an expression. Under such an assumption, one can define a translation τ_2 that optimizes an expression in this way:

$$\tau_2(v + v + v \times v + v \times v \times v + v) = v + v + v^{\wedge}e + v^{\wedge}e + v$$

Translation τ_2 transforms a product of factors v into a power with an exponent e . Later a semantic translation, not to be considered here, will assign appropriate values to the exponents e , in the example $e = 2$ and $e = 3$ from left to right.

Write a syntactic scheme (or grammar) G_2 that models translation τ_2 . Draw the translation tree of the sample expression above.

- (c) Are the schemes (or grammars) G_1 and G_2 deterministic? Explain your answers.
- (d) (optional) Resume the assumption of question (b), and now define a translation τ_3 that transforms a sum of terms v into one term with a multiplicative coefficient c , e.g., $\tau_3(v + v + v \times v + v) = c v + v \times v + v$ (later it will be assigned $c = 2$). Can you find a syntactic scheme (or grammar) G_3 that models translation τ_3 ? Should you change the source grammar? Explain your answer.

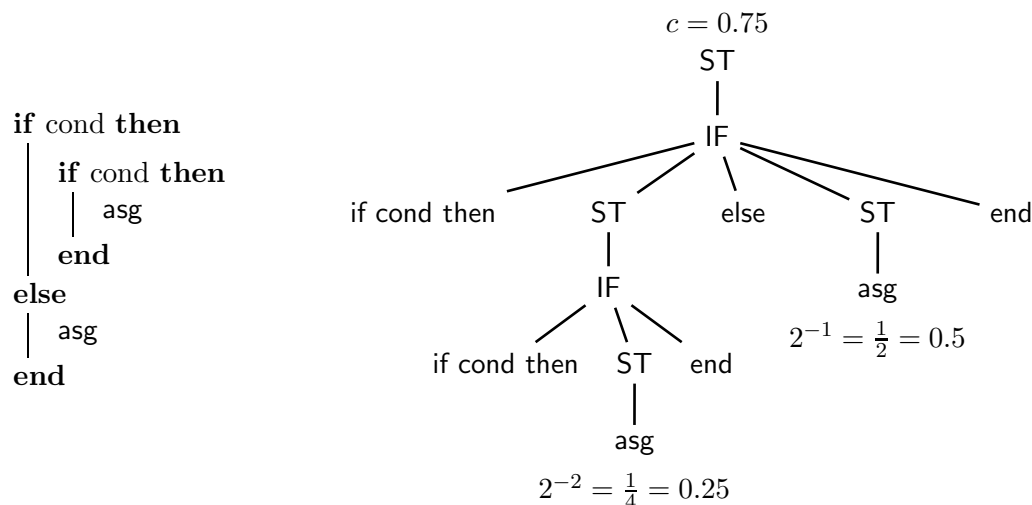
2. Consider the following (simplified) portion of a grammar of a programming language, with nested conditional statements `if ... then ... else ... end`, possibly without `else` branch (axiom `ST`):

$$\left\{ \begin{array}{l} 1: \langle \text{ST} \rangle \rightarrow \langle \text{IF} \rangle \\ 2: \langle \text{ST} \rangle \rightarrow \text{asg} \\ 3: \langle \text{IF} \rangle \rightarrow \text{if cond then } \langle \text{ST} \rangle \text{ else } \langle \text{ST} \rangle \text{ end} \\ 4: \langle \text{IF} \rangle \rightarrow \text{if cond then } \langle \text{ST} \rangle \text{ end} \end{array} \right.$$

We intend to compute a numerical parameter c that indicates the completeness degree of the nested instructions. Parameter c tells whether all the branches of the nested conditional instructions are complete, or whether some `else` branch is missing.

To do so, decreasing weights are assigned to the nested branches. Therefore, each instruction `asg` is assigned a value equal to 2^{-n} , where $n \geq 0$ is the number of enclosing conditional instructions.

For instance, in the sample program and tree below, the first `asg` (3-rd line) has a value $2^{-2} = \frac{1}{4} = 0.25$, while the second `asg` (6-th line) has a value $2^{-1} = \frac{1}{2} = 0.5$:



Answer the following questions (use the tables / trees / spaces on the next pages):

- Compute the correct value of parameter c for the entire conditional statement by using only one attribute, also named c , of a suitable type. The required result must be computed as the value of c in the root node of the abstract syntax tree. In the above example, the root node `ST` will have an attribute $c = 2^{-2} + 2^{-1} = 0.75$.
- Decorate the example tree on the next page and write the values of attribute c in all the relevant nodes.
- Say if the defined attribute grammar is of type L. Provide an explanation for your answer and avoid generic or tautological sentences.

attribute specification to be completed and semantic functions – question (a)

<i>name</i>	<i>type</i>	<i>domain</i>	<i>symbol</i>
<i>c</i>		real	

#	<i>syntax</i>	<i>semantics</i>
---	---------------	------------------

1: $ST_0 \rightarrow IF_1$

2: $ST_0 \rightarrow \text{asg}$

3: $IF_0 \rightarrow \text{if cond then } ST_1 \text{ else } ST_2 \text{ end}$

4: $IF_0 \rightarrow \text{if cond then } ST_1 \text{ end}$

syntax tree to be decorated – question (b)

